



HPC
HEIG-VD

Exploration de l'optimisation de code avec des outils LLM

Students:

Kilian Froidevaux
Basile Buxtorf

Teacher:

Alberto Dassatti
Bruno Da Rocha Carvalho

07.06.2026

Table of Contents

Introduction	3
Installation	3
Fonctionnement	4
Résultats	4
Conclusion	4

Introduction

Nous avons choisi le second challenge qui est un projet exploratoire sur l'optimisation de code avec des outils LLM.

Nous avons choisi d'implémenter un plugin d'optimisation de code pour Pi.dev, un harness d'agent IA très extensible. Ce plugin utiliserait des sous-agents spécialisés pour analyser le code, identifier les opportunités d'optimisation, proposer des améliorations et benchmarker les résultats. L'objectif est d'obtenir des gains de performance significatifs avec des modèles moins performants que les plus grands LLM en utilisant moins de tokens. Cela grâce à une approche plus ciblée où chaque sous-agent se concentre sur une tâche spécifique, et n'a accès qu'aux informations nécessaires à sa tâche, ce qui permet de réduire le nombre de tokens utilisés et d'améliorer l'efficacité globale du processus.

Installation

Pour installer Pi.dev, il y a plusieurs possibilités, toutes décrites sur leur site web. IL faut ensuite une clé d'API pour pouvoir utiliser un LLM. Nous avons choisi d'utiliser les modèles Kimi de Moonshot AI, qui sont très performants et offrent un bon rapport qualité-prix. Pour obtenir une clé d'API Kimi il faut suivre les instructions à <https://platform.kimi.ai/console/api-keys>.

Pour l'accès au modèle, il est aussi possible d'utiliser GitHub Copilot. En l'occurrence, les étudiants de la HEIG-VD ont accès à Copilot gratuitement, ce qui en fait une option intéressante même si avec <https://github.blog/news-insights/company-news/github-copilot-is-moving-to-usage-based-billing/> [les changements de tarification récents de GitHub Copilot], l'option a perdu de son attrait.

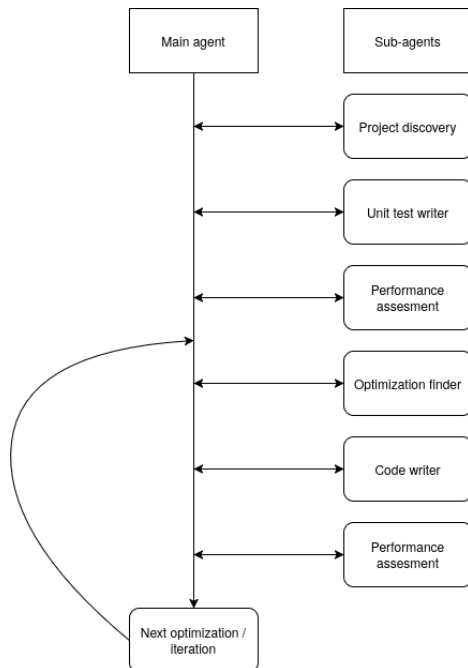
Ensuite, pour installer notre plugin d'optimisation de code, il suffit d'exécuter la commande suivante dans le terminal:

TODO:

Put actual repo link here

```
pi install git:github.com/user/repo
```

Fonctionnement



L'agent principal sert à orchestrer les interactions entre les différents sous-agents et à s'assurer que la pipeline reste cohérente.

Chaque sous-agent a une tâche spécifique et n'a accès qu'aux informations nécessaires à sa tâche. Lorsqu'un sous-agent termine sa tâche, l'agent principal reçoit les résultats et décide de la suite à donner.

Cette chaîne d'optimisation permet de traiter un aspect à la fois, en se concentrant sur une optimisation spécifique à chaque itération. L'agent principal a une liste de tous les types d'optimisation et les parcourt une par une. Il y a un choix à faire si une optimisation est jugée pertinente ou non. Si elle est jugée pertinente, l'agent principal demande à l'agent de performance de benchmarker le code avant l'optimisation, puis demande à l'agent d'optimisation de proposer des améliorations et de les appliquer au code. Ensuite, il demande à l'agent de performance de benchmarker le code après l'optimisation et compare les résultats. Si les résultats sont satisfaisants, il commit les changements sur la branche courante avec un message décrivant l'itération et le type d'optimisation, les performances avant et après, et les fichiers changés. Si les résultats ne sont insatisfaisants, il annule les changements et passe à l'optimisation suivante.

Résultats

Conclusion